Laravel 実装設計書

Loggit

Version 1.0　　作成日：2026 年 2 月 23 日

1. ディレクトリ構成

```
app/
├── Console/
│   ├── Commands/
│   │   └── AutoStopActivityLogs.php   # 24 時間自動停止バッチ
│   └── Kernel.php                     # スケジューラ設定
├── Exceptions/
│   ├── AlreadyTrackingException.php   # 同時計測中エラー
│   ├── AlreadyStoppedException.php    # 重複停止エラー
│   ├── TrackingInProgressException.php # 計測中への配賦エラー
│   └── TaskHierarchyException.php     # 階層超過エラー
├── Http/
│   ├── Controllers/API/
│   │   ├── AuthController.php
│   │   ├── ProjectController.php
│   │   ├── ProjectStatusController.php
│   │   ├── TaskController.php
│   │   ├── ActivityLogController.php
│   │   ├── TaskTimeLogController.php
│   │   └── UserController.php
│   ├── Requests/
│   │   ├── Auth/
│   │   │   ├── RegisterRequest.php
│   │   │   └── LoginRequest.php
│   │   ├── Project/
│   │   │   ├── StoreProjectRequest.php
│   │   │   └── UpdateProjectRequest.php
│   │   ├── Task/
│   │   │   ├── StoreTaskRequest.php
│   │   │   └── UpdateTaskRequest.php
│   │   ├── ActivityLog/
│   │   │   ├── StartActivityLogRequest.php
│   │   │   └── UpdateActivityLogRequest.php
│   │   └── TaskTimeLog/
│   │       └── StoreTaskTimeLogRequest.php
│   └── Resources/
│       ├── ProjectResource.php
│       ├── ProjectStatusResource.php
│       ├── TaskResource.php
│       ├── ActivityLogResource.php
│       ├── TaskTimeLogResource.php
│       └── SummaryResource.php
├── Models/
│   ├── User.php
│   ├── Project.php
│   ├── ProjectStatus.php
│   ├── Task.php            # SoftDeletes 使用
│   ├── ActivityLog.php
```

```
│   ├── TaskTimeLog.php
│   └── UserSetting.php
├── Policies/
│   ├── ProjectPolicy.php
│   ├── TaskPolicy.php
│   ├── ActivityLogPolicy.php
│   └── TaskTimeLogPolicy.php
└── Services/
    ├── ActivityLogService.php
    ├── TaskService.php
    ├── StatusService.php
    └── SummaryService.php
```

2. ルーティング（routes/api.php）

```
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login',    [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);

    // プロジェクト
    Route::apiResource('projects', ProjectController::class);
    Route::get('projects/{project}/statuses',
        [ProjectStatusController::class, 'index']);
    Route::put('projects/{project}/statuses/{status}',
        [ProjectStatusController::class, 'update']);

    // タスク
    Route::get('tasks', [TaskController::class, 'index']);
    Route::get('projects/{project}/tasks',
        [TaskController::class, 'projectIndex']);
    Route::post('projects/{project}/tasks',
        [TaskController::class, 'store']);
    Route::get('tasks/{task}',    [TaskController::class, 'show']);
    Route::put('tasks/{task}',    [TaskController::class, 'update']);
    Route::delete('tasks/{task}', [TaskController::class, 'destroy']);
    Route::post('tasks/{task}/restore',
        [TaskController::class, 'restore'])->withTrashed();

    // 活動ログ
    Route::get('activity-logs',         [ActivityLogController::class, 'index']);
    Route::get('activity-logs/summary',  [ActivityLogController::class,
'summary']);
    Route::post('activity-logs/start',   [ActivityLogController::class, 'start']);
    Route::post('activity-logs/{log}/stop',
        [ActivityLogController::class, 'stop']);
    Route::post('activity-logs/{log}/acknowledge',
        [ActivityLogController::class, 'acknowledge']);
    Route::put('activity-logs/{log}',    [ActivityLogController::class, 'update']);
    Route::delete('activity-logs/{log}', [ActivityLogController::class,
'destroy']);

    // タスク時間配賦
    Route::get('tasks/{task}/time-logs',
        [TaskTimeLogController::class, 'index']);
    Route::post('tasks/{task}/time-logs',
        [TaskTimeLogController::class, 'store']);
    Route::delete('time-logs/{timeLog}',
        [TaskTimeLogController::class, 'destroy']);

    // ユーザー設定
    Route::get('settings',          [UserController::class, 'settings']);
```

```
    Route::put('settings',          [UserController::class, 'updateSettings']);
    Route::put('user',              [UserController::class, 'update']);
    Route::put('user/password',     [UserController::class, 'updatePassword']);
});
```

3. Eloquent モデル定義

3.1 User

```
class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;

    protected $fillable = ['name', 'email', 'password'];
    protected $hidden   = ['password', 'remember_token'];
    protected $casts    = ['password' => 'hashed'];

    public function projects(): HasMany {
        return $this->hasMany(Project::class);
    }
    public function activityLogs(): HasMany {
        return $this->hasMany(ActivityLog::class);
    }
    public function setting(): HasOne {
        return $this->hasOne(UserSetting::class);
    }
}
```

3.2 Project

```
class Project extends Model
{
    protected $fillable = ['user_id', 'name', 'description', 'color', 'due_date'];
    protected $casts    = ['due_date' => 'date'];

    public function user(): BelongsTo {
        return $this->belongsTo(User::class);
    }
    public function statuses(): HasMany {
        return $this->hasMany(ProjectStatus::class)->orderBy('order');
    }
    public function tasks(): HasMany {
        return $this->hasMany(Task::class);
    }
    public function activityLogs(): HasMany {
        return $this->hasMany(ActivityLog::class);
    }

    // タスク数（完了・全件）
    public function getTaskCountAttribute(): int {
        return $this->tasks()->count();
    }
    public function getCompletedTaskCountAttribute(): int {
        return $this->tasks()
            ->whereHas('status', fn($q) => $q->where('order', 3))->count();
    }
}
```

3.3 ProjectStatus

```
class ProjectStatus extends Model
{
    protected $fillable = ['project_id', 'name', 'order'];

    public function project(): BelongsTo {
        return $this->belongsTo(Project::class);
    }
    public function tasks(): HasMany {
        return $this->hasMany(Task::class, 'status_id');
    }
}
```

3.4 Task

```
class Task extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'project_id', 'parent_task_id', 'status_id',
        'title', 'description', 'priority', 'due_date',
    ];
    protected $casts = ['due_date' => 'date', 'priority' => 'integer'];

    public function project(): BelongsTo {
        return $this->belongsTo(Project::class);
    }
    public function status(): BelongsTo {
        return $this->belongsTo(ProjectStatus::class, 'status_id');
    }
    public function parent(): BelongsTo {
        return $this->belongsTo(Task::class, 'parent_task_id');
    }
    public function children(): HasMany {
        return $this->hasMany(Task::class, 'parent_task_id');
    }
    public function timelogs(): HasMany {
        return $this->hasMany(TaskTimeLog::class);
    }

    // 優先度ラベル変換
    public function getPriorityLabelAttribute(): string {
        return match($this->priority) {
            1 => 'low', 2 => 'mid', 3 => 'high', default => 'low',
        };
    }
    // 累計配賦時間
    public function getTotalTrackedMinutesAttribute(): int {
        return $this->timelogss()->sum('duration_minutes');
    }
}
```

3.5 ActivityLog

```
class ActivityLog extends Model
{
    protected $fillable = [
        'user_id', 'project_id', 'started_at', 'ended_at',
        'note', 'auto_stopped', 'acknowledged_at',
```

```
    ];
    protected $casts = [
        'started_at'      => 'datetime',
        'ended_at'        => 'datetime',
        'acknowledged_at' => 'datetime',
        'auto_stopped'    => 'boolean',
    ];

    public function user(): BelongsTo {
        return $this->belongsTo(User::class);
    }
    public function project(): BelongsTo {
        return $this->belongsTo(Project::class);
    }
    public function taskTimeLogs(): HasMany {
        return $this->hasMany(TaskTimeLog::class);
    }

    // 総活動時間（分）
    public function getDurationMinutesAttribute(): ?int {
        if (!$this->ended_at) return null;
        return (int) $this->started_at->diffInMinutes($this->ended_at);
    }
    // 計測中フラグ
    public function getIsTrackingAttribute(): bool {
        return is_null($this->ended_at);
    }
}
```

3.6 TaskTimeLog

```
class TaskTimeLog extends Model
{
    protected $fillable = [
        'task_id', 'activity_log_id', 'duration_minutes', 'worked_on', 'note',
    ];
    protected $casts = ['worked_on' => 'date'];

    public function task(): BelongsTo {
        return $this->belongsTo(Task::class)->withTrashed();
    }
    public function activityLog(): BelongsTo {
        return $this->belongsTo(ActivityLog::class);
    }
}
```

4. マイグレーション定義

4.1 users

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('password');
    $table->timestamps();
});
```

4.2 user_settings

```
Schema::create('user_settings', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->unique()->constrained()->cascadeOnDelete();
    $table->boolean('enable_task_time_tracking')->default(true);
    $table->timestamps();
});
```

4.3 projects

```
Schema::create('projects', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->string('name');
    $table->text('description')->nullable();
    $table->string('color', 7)->nullable();
    $table->date('due_date')->nullable();
    $table->timestamps();
    $table->index('user_id');
});
```

4.4 project_statuses

```
Schema::create('project_statuses', function (Blueprint $table) {
    $table->id();
    $table->foreignId('project_id')->constrained()->cascadeOnDelete();
    $table->string('name', 100);
    $table->unsignedInteger('order');
    $table->timestamps();
    $table->index('project_id');
});
```

4.5 tasks

```
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->foreignId('project_id')->constrained()->cascadeOnDelete();
    $table->foreignId('parent_task_id')->nullable()
          ->constrained('tasks')->cascadeOnDelete();
    $table->foreignId('status_id')
          ->constrained('project_statuses')->restrictOnDelete();
    $table->string('title');
    $table->text('description')->nullable();
    $table->tinyInteger('priority')->default(1);
    $table->date('due_date')->nullable();
    $table->softDeletes();
    $table->timestamps();
    $table->index('project_id');
    $table->index('status_id');
    $table->index('due_date');
});
```

4.6 activity_logs

```
Schema::create('activity_logs', function (Blueprint $table) {
    $table->id();
```

```
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->foreignId('project_id')->nullable()
          ->constrained()->nullOnDelete();
    $table->datetime('started_at');
    $table->datetime('ended_at')->nullable();
    $table->text('note')->nullable();
    $table->boolean('auto_stopped')->default(false);
    $table->timestamp('acknowledged_at')->nullable();
    $table->timestamps();
    $table->index('user_id');
    $table->index('project_id');
    $table->index('started_at');
    $table->index(['user_id', 'started_at']);
});

// CHECK 制約（MySQL 8.0.16 以上）
DB::statement(
    'ALTER TABLE activity_logs
     ADD CONSTRAINT chk_ended_after_started
     CHECK (ended_at IS NULL OR ended_at > started_at)'
);
```

4.7 task_time_logs

```
Schema::create('task_time_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('task_id')->constrained()->cascadeOnDelete();
    $table->foreignId('activity_log_id')->nullable()
          ->constrained('activity_logs')->nullOnDelete();
    $table->unsignedInteger('duration_minutes');
    $table->date('worked_on')->nullable();
    $table->text('note')->nullable();
    $table->timestamps();
    $table->index('task_id');
    $table->index('activity_log_id');
});
```

5. FormRequest（バリデーション定義）

5.1 RegisterRequest

```
public function rules(): array
{
    return [
        'name'     => ['required', 'string', 'max:255'],
        'email'    => ['required', 'email', 'max:255', 'unique:users'],
        'password' => ['required', 'string', 'min:8'],
    ];
}
```

5.2 StoreProjectRequest

```
public function rules(): array
{
    return [
        'name'        => ['required', 'string', 'max:255'],
        'description' => ['nullable', 'string'],
```

```
        'color'       => ['nullable', 'string', 'regex:/^#[0-9A-Fa-f]{6}$/'],
        'due_date'    => ['nullable', 'date_format:Y-m-d'],
    ];
}
```

5.3 StoreTaskRequest

```
public function rules(): array
{
    return [
        'title'          => ['required', 'string', 'max:255'],
        'description'    => ['nullable', 'string'],
        'status_id'      => ['required', 'integer', 'exists:project_statuses,id'],
        'priority'       => ['nullable', 'integer', 'in:1,2,3'],
        'due_date'       => ['nullable', 'date_format:Y-m-d'],
        'parent_task_id' => ['nullable', 'integer', 'exists:tasks,id'],
    ];
}
```

5.4 StartActivityLogRequest

```
public function rules(): array
{
    return [
        'project_id' => ['nullable', 'integer', 'exists:projects,id'],
        'note'       => ['nullable', 'string'],
    ];
}
```

5.5 StoreTaskTimeLogRequest

```
public function rules(): array
{
    return [
        'duration_minutes' => ['required', 'integer', 'min:1'],
        'activity_log_id'  => ['nullable', 'integer', 'exists:activity_logs,id'],
        'worked_on'        => ['nullable', 'date_format:Y-m-d'],
        'note'             => ['nullable', 'string'],
    ];
}
```

6. Policy クラス定義

6.1 ProjectPolicy

```
class ProjectPolicy
{
    public function view(User $user, Project $project): bool {
        return $user->id === $project->user_id;
    }
    public function update(User $user, Project $project): bool {
        return $user->id === $project->user_id;
    }
    public function delete(User $user, Project $project): bool {
        return $user->id === $project->user_id;
```

```
    }
}
```

6.2 TaskPolicy

```
class TaskPolicy
{
    public function view(User $user, Task $task): bool {
        return $user->id === $task->project->user_id;
    }
    public function update(User $user, Task $task): bool {
        return $user->id === $task->project->user_id;
    }
    public function delete(User $user, Task $task): bool {
        return $user->id === $task->project->user_id;
    }
    public function restore(User $user, Task $task): bool {
        return $user->id === $task->project->user_id;
    }
}
```

6.3 ActivityLogPolicy

```
class ActivityLogPolicy
{
    public function view(User $user, ActivityLog $log): bool {
        return $user->id === $log->user_id;
    }
    public function update(User $user, ActivityLog $log): bool {
        return $user->id === $log->user_id;
    }
    public function delete(User $user, ActivityLog $log): bool {
        return $user->id === $log->user_id;
    }
}
```

7. Service クラス定義

7.1 ActivityLogService

```
class ActivityLogService
{
    public function start(User $user, array $data): ActivityLog
    {
        return DB::transaction(function () use ($user, $data) {
            $active = ActivityLog::where('user_id', $user->id)
                ->whereNull('ended_at')
                ->lockForUpdate()
                ->first();

            if ($active) {
                throw new AlreadyTrackingException();
            }

            return ActivityLog::create([
                'user_id'    => $user->id,
                'project_id' => $data['project_id'] ?? null,
```

```
                'note'       => $data['note'] ?? null,
                'started_at' => now(),
            ]);
        });
    }

    public function stop(ActivityLog $log): ActivityLog
    {
        return DB::transaction(function () use ($log) {
            $log = ActivityLog::lockForUpdate()->find($log->id);

            if ($log->ended_at !== null) {
                throw new AlreadyStoppedException();
            }

            $log->update([
                'ended_at'     => now(),
                'auto_stopped' => false,
            ]);

            return $log->fresh();
        });
    }
}
```

7.2 TaskService

```
class TaskService
{
    public function getDepth(int $parentTaskId): int
    {
        $depth = 1;
        $currentId = $parentTaskId;

        while ($currentId !== null) {
            $task = Task::withTrashed()->find($currentId);
            if (!$task) break;
            $currentId = $task->parent_task_id;
            $depth++;
            if ($depth > 3) break; // 無限ループ防止
        }

        return $depth;
    }

    public function checkCircularReference(int $taskId, int $parentId): bool
    {
        if ($taskId === $parentId) return true;

        $currentId = $parentId;
        while ($currentId !== null) {
            $task = Task::find($currentId);
            if (!$task) break;
            if ($task->parent_task_id === $taskId) return true;
            $currentId = $task->parent_task_id;
        }

        return false;
    }

    public function softDeleteWithChildren(Task $task): void
    {
        $task->children()->each(fn($child) =>
```

```
            $this->softDeleteWithChildren($child)
        );
        $task->delete();
    }

    public function restoreWithChildren(Task $task): void
    {
        $task->restore();
        $task->children()->withTrashed()->each(fn($child) =>
            $this->restoreWithChildren($child)
        );
    }
}
```

7.3 StatusService

```
class StatusService
{
    private const MAX_STATUSES = 3;
    private const MIN_STATUSES = 1;

    public function canAdd(Project $project): bool
    {
        return $project->statuses()->count() < self::MAX_STATUSES;
    }

    public function canDelete(Project $project): bool
    {
        return $project->statuses()->count() > self::MIN_STATUSES;
    }

    public function createDefaults(Project $project): void
    {
        $defaults = [
            ['name' => 'Todo',        'order' => 1],
            ['name' => 'In Progress', 'order' => 2],
            ['name' => 'Done',        'order' => 3],
        ];

        foreach ($defaults as $status) {
            $project->statuses()->create($status);
        }
    }
}
```

7.4 SummaryService

```
class SummaryService
{
    public function getSummary(User $user, string $period): array
    {
        [$start, $end] = $this->getPeriodRange($period);

        // 総活動時間（activity_logs ベース）
        $totalActivity = ActivityLog::where('user_id', $user->id)
            ->whereNotNull('ended_at')
            ->whereBetween('started_at', [$start, $end])
            ->selectRaw('SUM(TIMESTAMPDIFF(MINUTE, started_at, ended_at)) as
total')
            ->value('total') ?? 0;
```

```
        // 総配賦時間（task_time_logs ベース）
        $totalTracked = TaskTimeLog::whereHas('task', fn($q) =>
            $q->where('user_id', $user->id)  // tasks→projects→user_id
        )->whereBetween('worked_on', [$start->toDateString(), $end-
>toDateString()])
            ->sum('duration_minutes');

        // 日別集計
        $daily = $this->getDailyBreakdown($user, $start, $end);

        // プロジェクット別集計
        $byProject = $this->getByProjectBreakdown($user, $start, $end);

        return compact('totalActivity', 'totalTracked', 'daily', 'byProject');
    }

    private function getPeriodRange(string $period): array
    {
        $now = Carbon::now('Asia/Tokyo');
        if ($period === 'week') {
            $start = $now->copy()->startOfWeek(Carbon::SUNDAY)->utc();
            $end   = $now->copy()->endOfWeek(Carbon::SATURDAY)->utc();
        } else {
            $start = $now->copy()->startOfMonth()->utc();
            $end   = $now->copy()->endOfMonth()->utc();
        }
        return [$start, $end];
    }
}
```

8. Controller クラス定義

8.1 ActivityLogController（主要メソッド）

```
class ActivityLogController extends Controller
{
    public function __construct(
        private ActivityLogService $service
    ) {}

    public function start(StartActivityLogRequest $request): JsonResponse
    {
        $log = $this->service->start(
            auth()->user(),
            $request->validated()
        );
        return new ActivityLogResource($log)
            ->response()->setStatusCode(201);
    }

    public function stop(ActivityLog $log): JsonResponse
    {
        $this->authorize('update', $log);
        $stopped = $this->service->stop($log);
        return (new ActivityLogResource($stopped))->response();
    }

    public function summary(Request $request): JsonResponse
    {
        $period = $request->query('period', 'week');
        $data   = $this->summaryService->getSummary(auth()->user(), $period);
```

```
        return (new SummaryResource($data))->response();
    }
}
```

8.2 TaskController（主要メソッド）

```
class TaskController extends Controller
{
    public function __construct(
        private TaskService $service
    ) {}

    public function store(StoreTaskRequest $request, Project $project):
JsonResponse
    {
        $this->authorize('update', $project);
        $data = $request->validated();

        // 階層チェック
        if (isset($data['parent_task_id'])) {
            $depth = $this->service->getDepth($data['parent_task_id']);
            if ($depth >= 3) throw new TaskHierarchyException();

            // 循環参照チェック（更新時のみ）
        }

        $task = $project->tasks()->create($data);
        return (new TaskResource($task->load('status', 'children')))
            ->response()->setStatusCode(201);
    }

    public function destroy(Task $task): Response
    {
        $this->authorize('delete', $task);
        $this->service->softDeleteWithChildren($task);
        return response()->noContent();
    }

    public function restore(Task $task): JsonResponse
    {
        $this->authorize('restore', $task);
        $this->service->restoreWithChildren($task);
        return (new TaskResource($task->fresh()))->response();
    }
}
```

9. カスタム例外クラス

9.1 例外クラス一覧

クラス名	ステータス	message
AlreadyTrackingException	422	すでに計測中の活動ログがあります
AlreadyStoppedException	422	この活動ログはすでに停止されています
TrackingInProgressException	422	計測中の活動ログには配賦できません

クラス名	ステータス	message
TaskHierarchyException	422	サブタスクは 3 階層までしか作成できません

9.2 例外クラス実装例

```
class AlreadyTrackingException extends RuntimeException
{
    public function render(): JsonResponse
    {
        return response()->json([
            'message' => 'すでに計測中の活動ログがあります',
            'errors'  => [],
        ], 422);
    }
}
```

10. バッチ・スケジューラ

10.1 AutoStopActivityLogs コマンド

```
class AutoStopActivityLogs extends Command
{
    protected $signature   = 'activitylogs:auto-stop';
    protected $description = '24 時間以上計測中のログを自動停止する';

    public function handle(): void
    {
        $targets = ActivityLog::whereNull('ended_at')
            ->where('started_at', '<=', now()->subHours(24))
            ->get();

        foreach ($targets as $log) {
            $log->update([
                'ended_at'     => $log->started_at->addHours(24),
                'auto_stopped' => true,
            ]);
        }

        $this->info("Auto-stopped {$targets->count()} logs.");
    }
}
```

10.2 Kernel.php スケジューラ設定

```
protected function schedule(Schedule $schedule): void
{
    $schedule->command('activitylogs:auto-stop')
             ->hourly()
             ->withoutOverlapping();
}
```

*※ cron 設定: * * * * * php /path/to/artisan schedule:run >> /dev/null 2>&1*

11. API リソースクラス（レスポンス整形）

11.1 ActivityLogResource

```
class ActivityLogResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'                 => $this->id,
            'project_id'         => $this->project_id,
            'project_name'       => $this->project?->name,
            'started_at'         => $this->started_at->setTimezone('Asia/Tokyo')
                                        ->toIso8601String(),
            'ended_at'           => $this->ended_at?->setTimezone('Asia/Tokyo')
                                        ->toIso8601String(),
            'duration_minutes'   => $this->duration_minutes,
            'note'               => $this->note,
            'auto_stopped'       => $this->auto_stopped,
            'is_tracking'        => $this->is_tracking,
            'task_time_logs'     => TaskTimeLogResource::collection(
                $this->whenLoaded('taskTimeLogs')
            ),
            'created_at'         => $this->created_at->setTimezone('Asia/Tokyo')
                                        ->toIso8601String(),
        ];
    }
}
```

11.2 TaskResource

```
class TaskResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'                        => $this->id,
            'project_id'                => $this->project_id,
            'project_name'              => $this->project?->name,
            'parent_task_id'            => $this->parent_task_id,
            'status'                    => new ProjectStatusResource($this->status),
            'title'                     => $this->title,
            'description'               => $this->description,
            'priority'                  => $this->priority,
            'priority_label'            => $this->priority_label,
            'due_date'                  => $this->due_date?->format('Y-m-d'),
            'subtasks'                  => TaskResource::collection(
                $this->whenLoaded('children')
            ),
            'subtask_count'             => $this->children()->count(),
            'completed_subtask_count'   => $this->children()
                ->whereHas('status', fn($q) => $q->where('order', 3))->count(),
            'total_tracked_minutes'     => $this->total_tracked_minutes,
            'deleted_at'                =>
$this->deleted_at?->setTimezone('Asia/Tokyo')
                                            ->toIso8601String(),
            'created_at'                =>
$this->created_at->setTimezone('Asia/Tokyo')
                                            ->toIso8601String(),
            'updated_at'                =>
$this->updated_at->setTimezone('Asia/Tokyo')
                                            ->toIso8601String(),
        ];
```

```
    }
}
```

12. レイヤー責務分担まとめ

クラス	責務
FormRequest	入力値のバリデーション（型・必須・形式チェック）のみ担当。ビジネスロジックは持たない
Controller	リクエスト受付・Policy による権限確認・Service の呼び出し・Resource によるレスポンス返却
Policy	リソースの所有権チェックのみ担当（user_id の一致確認）
Service	ビジネスロジック全般（トランザクション・業務ルールバリデーション・複数モデルの操作）
Model	DB とのやり取り・リレーション定義・アクセサ（計算プロパティ）
Resource	レスポンス JSON の整形・タイムゾーン変換・不要フィールドの除外
Exception	業務ルール違反を表す例外クラス。render メソッドで 422 レスポンスを返す
Command	バッチ処理（24 時間自動停止）。Service を呼び出す形でロジックを分離